

Performance of Machine Learning Algorithms for Class-Imbalanced Process Fault Detection Problems

Taehyung Lee, Ki Bum Lee, and Chang Ouk Kim

Abstract—In recent years, the semiconductor manufacturing industry has recognized class imbalance as a major impediment to the development of high-performance fault detection (FD) models. Class imbalance refers to skews in class distribution in which normal wafer samples are considerably more abundant than fault samples. In such a situation, standard machine learning algorithms create FD models with classification boundaries that are biased toward majority-class data, resulting in high type II error rates. In this paper, we compare the performance of machine learning algorithms for class-imbalanced FD problems. We evaluate the performance of three sampling-based algorithms, four ensemble algorithms, four instance-based algorithms, and two support vector machine algorithms. Two experiments were conducted to compare algorithm performance using etching process data and chemical vapor deposition process data. Different data scenarios were considered by setting the imbalance ratio to three levels. The results of the experiments indicated that the instance-based algorithms presented excellent performance even when the imbalance ratio increased.

Index Terms—Class imbalance, fault detection, machine learning, performance evaluation, semiconductor manufacturing.

I. INTRODUCTION

SEMICONDUCTOR process faults are the main causes of wafer defects. A variety of machine learning-based fault detection (FD) models have been developed to minimize the time and cost for producing wafer scraps. In recent years, however, the semiconductor manufacturing industry has recognized class imbalance as a major impediment to the development of high-performance FD models. A two-class dataset is imbalanced when one of the classes (i.e., the minority class) is heavily overwhelmed in number by the other class (i.e., the majority class). In a skewed class distribution, standard classification algorithms, such as support vector machines (SVMs) and decision trees (DTs), create FD models with classification boundaries that are biased toward the

majority class that classify normal wafers perfectly but can misclassify defects, resulting in high type II error rates. This issue is particularly important in industry because it is costly to misclassify rare wafers that are labeled as defective (i.e., the minority class).

In a previous study [1], we compared the performance of FD models under the assumption that the class distribution of the data to train FD models is balanced. Standard machine learning algorithms were used to construct the FD models. In this study, we construct FD models with machine learning algorithms that are appropriate for imbalanced situations, compare the performance of the models, and discuss the advantages and disadvantages of the algorithms.

Imbalance learning can be classified into sampling-based, ensemble, instance-based, and SVM family approaches. In this study, we evaluated the performance of three sampling-based algorithms, four ensemble algorithms, four instance-based algorithms, and two SVM algorithms. A sampling-based approach equalizes the volume of data within two classes; thus, the synthetic minority oversampling technique (SMOTE) [2], the neighborhood cleaning rule (NCL) [3], and a random oversampling are tested in this study. Each sampling algorithm is combined with a DT or SVM to construct an FD model. The ensemble approach generates training sample groups with the same volume from a training dataset using sampling with replacement, trains a weak classifier for each sample group, and combines the classification results of the classifiers to suggest a final output. The ensemble algorithms tested in this study are the Bagging [4], boosting [5], random forest [6], and SMOTEBoost [7] algorithms. The DT and SVM algorithms are investigated as weak classifiers in this study.

The instance-based algorithms tested in this study include FD using the kNN rule (FD-kNN) [8], principal component based kNN (PC-kNN) [9], adaptive Mahalanobis distance based kNN (MD-kNN) [10], and incremental clustering based FD method (IC-FDM) [11]. In addition, this study tested the cost-sensitive SVM (CS-SVM) algorithm [12] and the one-class SVM (OC-SVM) algorithm [13]. The cost-sensitive approach resolves imbalanced situations by imposing a large penalty on classification errors in minority class samples. The one-class SVM learns the boundary of the majority class distribution in an unsupervised manner. The instance-based algorithms and OC-SVM use only normal data to preform FD; therefore, these algorithms are referred to as one-class learning algorithms.

Manuscript received April 18, 2016; revised July 16, 2016; accepted August 18, 2016. Date of publication August 24, 2016; date of current version October 27, 2016. This work was supported in part by the Technology Innovation Program (Development of Big Data-Based Analysis and Control Platform for Semiconductor Manufacturing Plants) through the Ministry of Trade, Industry and Energy, South Korea, under Grant 10045913, in part by the Global Ph.D. Fellowship Program through the National Research Foundation of Korea (NRF) by the Ministry of Education under Grant NRF-2015H1A2A1031081, and in part by the NRF Grant by the Korean Government (MSIP) under Grant NRF-2016R1A2B4008337.

The authors are with the Department of Information and Industrial Engineering, Yonsei University, Seoul 120-749, South Korea (e-mail: th.lee@kia.com; kiblee@yonsei.ac.kr; kimco@yonsei.ac.kr).

Digital Object Identifier 10.1109/TSM.2016.2602226

TABLE I
FD MODELS TESTED IN THIS STUDY FOR
THE CLASS IMBALANCE PROBLEM

Family	FD model	Abbreviation
Sampling-based algorithms (F1)	SMOTE with DT or SVM,	SDT, SSVM
	NCL with DT or SVM,	NDT, NSVM
	Random oversampling with DT or SVM	RODT, ROSVM
Ensemble algorithms (F2)	Bagging with DT or SVM,	BDT, BSVM
	AdaBoost with DT or SVM,	ABDT, ABSVM
	SMOTEBoost with DT or SVM,	SBDT, SBSVM
	Random forest	RF
Instance-based algorithms (F3)	FD using the kNN rule,	FD-kNN
	Principal component-based kNN rule,	PC-kNN
	Adaptive Mahalanobis distance and kNN rule,	MD-kNN
	Incremental clustering-based FD method	IC-FDM
SVM Family (F4)	Cost sensitive SVM,	CS-SVM
	One class SVM	OC-SVM

We conducted two experiments to evaluate algorithm performance using etching process data in the first experiment and chemical vapor deposition (CVD) process data in the second experiment. In each experiment, we created data scenarios by considering three levels of the imbalance ratio (IR), defined as the number of majority class instances divided by the number of minority class instances. We prepared a total of 50 datasets for each scenario. For each data scenario, the abovementioned FD models are tested with the 50 datasets, and the average outputs are compared. We used the G-mean, F-measure, and area under the curve (AUC) to compare the model performances. These measures are appropriate for class-imbalanced situations because they consider the type I and type II errors simultaneously in the performance evaluation of learning models.

The remainder of this paper is organized as follows. Section II briefly explains the principles of the FD algorithms for use with imbalanced data. Sections III and IV present the results of the experiments and discuss their significance. Finally, Section V provides the conclusion of this study.

II. LEARNING ALGORITHMS FOR IMBALANCED DATA

As shown in Table I, a total of nineteen FD models were investigated in this study.

A. Sampling-Based Algorithms

1) *Random Sampling*: Random sampling increases or decreases the amount of data in a class by selecting random samples with replacement from the class dataset. In this study, we test oversampling, which is used to increase the amount of samples in the minority class. Although this algorithm simply mitigates the class imbalance, model overfitting is likely to occur due to the possibility of the same data being repeatedly sampled.

2) *SMOTE*: This algorithm is an oversampling technique that generates synthetic data from a source dataset. The SMOTE algorithm is relatively free from overfitting compared

to random sampling because the same data cannot be selected repeatedly. The algorithm generates new data by combining a minority class sample with its nearest neighbor. When a minority class sample x_i has k minority neighbors, a synthetic sample is generated as follows:

$$x_{new} = x_i + (\hat{x}_i - x_i) \times \delta. \quad (1)$$

In (1), $\hat{x}_i$ is a sample that was selected randomly among its k neighbors and δ is a random number in the interval $[0, 1]$. The amount of synthetic data that must be generated is controlled by the oversampling ratio ν . The SMOTE algorithm is known to exhibit the best performance in the available oversampling algorithms. However, the features of the entire minority class population might not be reflected in the classifier learning because synthetic data are generated from the source dataset.

3) *NCL*: This algorithm is an edited nearest neighbor (ENN)-based data cleaning (i.e., under-sampling) algorithm. If the class of a sample is different from the representative class of its k neighbors, then the ENN algorithm considers the sample as noise and deletes the sample. Based on the ENN algorithm, the data-reduction process of the NCL algorithm consists of two steps. The first step runs the ENN algorithm with the majority class samples and removes any identified noise accordingly. The second step deletes any majority class samples in the k neighbors of a minority class sample if the representative class is the majority class. The NCL algorithm defines a clear border for classification. To prevent the excessive elimination of majority class samples, the second step is applied to only half of all of the minority class samples. The NCL algorithm generally cleans majority class samples within the minority class area. The user should specify the oversampling ratio in the SMOTE algorithm; however, the number of samples deleted in the NCL algorithm is determined based on the parameter k , which is the number of neighbors.

B. Ensemble Algorithms

These algorithms output a classification result by combining several weak classifiers. A weak classifier performs simple learning and is likely to be overfitted by the training data. When provided with a new sample, ensemble algorithms collect the classification results of weak classifiers and issue a generalized conclusion through a voting process. With regard to the ensemble algorithms, boosting (including SMOTEBoosting) has been widely applied to address class imbalance problems [14]. This is a bootstrap sampling-based iterative algorithm. In each iteration, a bootstrap sample set is created based on a class-imbalanced source dataset and a weak classifier is trained with the sample set. Bootstrap sampling is a type of random sampling with replacement. Therefore, minority class samples are likely to be misclassified in early iterations because the samples have a small chance to be selected for classifier training. However, as the iterations progress, the algorithm increases the sampling probability of misclassified samples, and this sampling strategy creates classifiers with a decreased misclassification ratio of the minority class samples. In addition, this study tested the

TABLE II
SVM-BASED BAGGING

Input: training set $X_{train} = \{x_i, y_i\}, i = 1, \dots, N; y_i \in \{-1, 1\}; T$: number of iterations; svm: SVM as a weak learner; q : Bootstrap size.
Output: bagged classifier: $H(x) = \text{sign}(\sum_{t=1}^T h_t(x))$ where h_t is an induced classifier (with $h_t(x) \in \{-1, 1\}$).
1: **for** $t = 1$ to T **do**
2: $h_t \leftarrow \text{svm}(\text{random_sampling_with_replacement}(X_{train}, q))$
3: **end for**

bagging method, which is a standard ensemble algorithm, and the random forest method, which is known to display excellent performance in many field problems, such as FD models.

1) *Bagging*: This algorithm uses bootstrap sampling to create multiple training sample groups. The bagging algorithm trains a weak classifier for each sample group. Table II describes the bagging algorithm that uses an SVM as the weak classifier. In Table II, bagging iterates T times and trains an SVM classifier h_t ($t = 1, \dots, T$) with a sample group in each iteration. The output of a new sample x is the result of the majority voting, which is described by $H(x) = \text{sign}(\sum_{t=1}^T h_t(x))$.

2) *Boosting*: In the bagging algorithm, source samples have an equal probability of being included in a training sample group due to bootstrap sampling. Conversely, the boosting algorithm is also an iterative algorithm and increases the sampling probability in each iteration for samples that have been misclassified in a previous iteration. AdaBoost is a popular boosting algorithm [15] and was originally designed to use a DT as the weak classifier. However, the boosting algorithm also uses an SVM as a weak classifier. Table III explains the principle of AdaBoost that uses an SVM as a weak classifier.

In Table III, AdaBoost updates the sampling distribution D_t for every iteration t . On the first iteration, a classifier is trained with samples drawn with a uniform distribution (i.e., lines 1–3). The sampling distribution at the next iteration is updated based on the misclassification probability ε_t , which increases the probability of the misclassified samples to be selected as training data (i.e., lines 4–10). A new classifier is then trained with the new training data. The conclusion $H(x)$ for a new sample x undergoes a voting process with the trained classifiers, where a high weight α_t is assigned to a classifier with a small misclassification probability ε_t .

3) *SMOTEBoost*: The classifiers generated by the boosting algorithm might not learn the minority class sufficiently well because the size of the source samples is limited. Thus, the boosting algorithm may still favor majority class samples in the classification. The SMOTEBoost algorithm is an expansion of the AdaBoost algorithm. The SMOTEBoost algorithm generates a synthetic minority class at each boosting iteration using the SMOTE oversampling technique and mitigates bias toward the majority class accordingly.

4) *Random Forest*: This algorithm is similar to the bagging algorithm in that they use DTs as weak classifiers. The bagging and random forest algorithms use a source dataset to generate training sample groups via bootstrap sampling. The difference is that the random forest algorithm generates the tree with

TABLE III
SVM-BASED ADABOOST

Input: training set $X_{train} = \{x_i, y_i\}, i = 1, \dots, N; y_i \in \{-1, 1\}; T$: number of iterations; svm: SVM as a weak learner.
Output: boosted classifier: $H(x) = \text{sign}(\sum_{t=1}^T \alpha_t h_t(x))$, where h_t is an induced classifier (with $h_t(x) \in \{-1, 1\}$) and α_t is the weight assigned for each classifier.
1: $D_1(i) \leftarrow \frac{1}{N}$ for $i = 1, \dots, N$
2: **for** $t = 1$ to T **do**
3: $h_t \leftarrow \text{svm}(\text{random_sampling_with_replacement}(X_{train}, D_t))$
4: $\varepsilon_t \leftarrow \sum_{i=1}^N D_t(i) [h_t(x_i) \neq y_i]$
5: $\alpha_t = \frac{1}{2} \ln \frac{1-\varepsilon_t}{\varepsilon_t}$
6: $D_{t+1}(i) = D_t(i) \cdot e^{\{-\alpha_t h_t(x_i) y_i\}}$ for $i = 1, \dots, N$
7: Renormalize D_{t+1}
8: **end for**

a smaller attribute set than the original attribute set. Generally, given that a sample has p attributes measured on them, the random forest algorithm creates a tree for each sample group using $\sqrt{p}$ attributes that have been randomly selected from the original attributes [6]. As a result, a different tree is generated even with similar data. Similar to the bagging algorithm, the random forest algorithm predicts the outcome of new instances using a voting process with trained DTs.

C. Instance-Based Algorithms

1) *FD-kNN*: This algorithm is an extension of the standard kNN rule, which determines the k neighbors of a new sample and assigns the representative class of the neighbors as the class of the sample. The FD-kNN algorithm conducts FD only with normal samples (i.e., majority class samples). Specifically, the FD-kNN algorithm assumes that the sum of the squared distances between normal samples in a Euclidean space follows a non-central chi-square distribution. For a new sample, the FD-kNN algorithm considers the sum of the squared distances from the sample to the k neighbors to be a test statistic. The FD-kNN algorithm compares the test statistic score with a threshold (e.g., a 95% confidence limit) of the non-central chi-square distribution and classifies the sample as normal if the score is smaller than the threshold. Otherwise, the sample is classified as defective.

2) *PC-kNN*: This algorithm is an extension of the FD-kNN algorithm and performs FD in a principal component space. Because the FD-kNN algorithm calculates distances to neighboring samples in the space consisting of input variables, the calculation becomes overloaded when many variables are present in a sample. To reduce the dimensionality, the PC-kNN algorithm calculates distances to neighbors in a principal component space, which reduces both the amount of memory required to perform the kNN algorithm and the computational load.

3) *MD-kNN*: The FD-kNN and PC-kNN algorithms use the Euclidean distance metric to identify k neighbors, whereas the MD-kNN algorithm uses the Mahalanobis distance metric, which considers the data density when identifying neighbors. When a new sample is introduced, the MD-kNN algorithm

first analyzes the data density near the sample by finding K neighbors based on the Euclidean distance and estimating the corresponding covariance matrix. Next, the algorithm identifies the k neighbors based on the Mahalanobis distance that reflects the covariance matrix. The test statistic is the sum of the squared Mahalanobis distances between the sample and its k neighbors. If the test statistic score is smaller than a confidence limit, the sample is classified as normal; otherwise, it is classified as defective.

4) *IC-FDM*: This algorithm groups only normal samples in the source dataset into small clusters with elliptical shapes and leaves a few defective samples un-clustered. The IC-FDM algorithm only uses the mean vectors and covariance matrices of the normal clusters to classify new samples. These two statistical measures determine the boundary of the normal clusters in the Euclidean space. Given a new sample, the algorithm determines whether the sample is within the boundary of the nearest normal cluster by calculating the Mahalanobis distance between the sample and the center of the normal cluster and comparing the distance with a threshold that is set based on an F-like distribution. When the new sample is classified as normal, the algorithm updates the mean vector and covariance matrix of the nearest cluster using the sample information.

D. SVM Family

1) *CS-SVM*: An SVM maximizes the margin of a hyperplane between support vectors to establish an optimized classification boundary. Although an SVM is successful when used with class-balanced datasets, it creates a hyperplane that is biased toward the majority class data in imbalanced problems. This occurs because classification accuracy, which is the objective of SVM learning, is affected by the amount of data considered; thus, classification boundaries that are biased toward the normal data, which account for the majority of the data, form. Thus, a correction method is used to assign a different penalty cost to the classification error of each class. The CS-SVM algorithm is an SVM that considers class-specific error costs in the objective function. The model is formulated as follows:

$$\begin{aligned} \underset{(w,b)}{\operatorname{argmin}} \quad & \frac{1}{2} \|w\|^2 + C_+ \sum_{i:y_i=1} \xi_i + C_- \sum_{j:y_j=-1} \xi_j \\ \text{subject to} \quad & y_i[(w^T x_i) + b] \geq 1 - \xi_i, \quad i = 1, \dots, n \end{aligned} \quad (2)$$

where (x, y) denotes a sample and ξ is a slack variable. An SVM uses a soft margin to form the classification boundary even though there is no hyperplane $w^T x + b$ that perfectly separates minority class samples from majority class samples. The soft margin is represented by introducing the slack variable ξ in the objective function. The CS-SVM algorithm imposes class-specific error costs on ξ . The costs C_+ and C_- are assigned to the normal and defective classes, respectively. The cost C_- is typically set higher than C_+ to increase the misclassification penalty of minority class samples.

2) *OC-SVM*: This algorithm builds an FD model utilizing only the majority class samples as training data.

The samples are divided into two sets, termed the objective field and non-objective field, and a model parameter, termed the ν parameter, is introduced to control the ratio of the training samples included in the non-objective field. The classification boundary identifies the surface of a minimal hypersphere that includes all data in the objective field. When a test sample lies inside the hypersphere, it is predicted to be a member of the majority class.

III. EXPERIMENT WITH ETCHING DATA

A. Data and Setup

In this experiment, we simulated a dry etcher LAM TCP 9600 to generate semiconductor process data [1]. By examining real processing data [16] that consisted of eighteen process parameters, we identified six core process parameters that introduce certain process faults (e.g., TCP load, BCl₃ flow, Cl₂ flow, pressure, He pressure, and RF load). The trajectories of the six process parameters were simulated by considering the covariance structure measured from the real data with random noise. A detailed description on the etching simulation was provided by Lee and Kim [1]. We extracted four structural features for each trajectory: the mean, standard deviation, maximum, and minimum. Therefore, the total number of variables that describe a wafer sample was 24. In a preceding experiment, Lee and Kim [1] demonstrated that structural features produced the best performance among the existing feature selection methods.

A single dataset includes 1,000 normal wafer samples. The number of defect samples included in the dataset differed based on the IR. We considered three IRs with a given number of normal samples over the number of defect samples; the latter value is shown in parentheses: 10 (100 defects), 100 (10 defects), and 1,000 (1 defect). A total of 50 datasets were prepared for each scenario.

The parameters of the algorithms discussed above were set as follows. For the SMOTE algorithm, the oversampling ratio ν was set differently depending on the IR such that the numbers of normal and defect samples were equal; the parameter k was set to five [2]. The parameter k of the NCL algorithm was set to three [3]. The iteration count T of the bagging and AdaBoost algorithms was set equal to 50 [7]. The bootstrap size of the bagging algorithm was set equal to the number of training samples. The number of trees in the random forest algorithm was set equal to 100. The parameters of the SMOTEBoost algorithm were set equal to the parameters of the SMOTE and AdaBoost algorithms. The number of neighbors k of the FD-kNN and PC-kNN algorithms was set equal to ten, and the confidence level α was set equal to 0.95 [8]. The parameter k of the MD-kNN algorithm was set equal to that of the FD-kNN algorithm; however, the parameter K required to calculate the covariance was set equal to 100 [10]. The parameters of the IC-FDM algorithm were set equal to the values used in reference [11]. In the CS-SVM algorithm, the misclassification costs of the normal and defective classes were set based on the IR. For example, if the IR were equal to ten, a tenfold cost would be assigned to the

defective data. Finally, all algorithms were implemented in the R programming language [17].

B. Performance Measures

Because the total classification accuracy leads to a biased result toward the majority class, it is not an appropriate performance measure for an imbalanced problem. In the experiments of this study, we used the G-mean, F-measure, and AUC as performance measures. Here, the majority class that is labeled as normal is denoted by “negative”, and the minority class that is labeled as defective is denoted by “positive”. True positive (TP) and true negative (TN) were thus defined as the number of samples that were correctly classified as defective and normal, respectively. Thus, false positive (FP) and false negative (FN) were defined as the amount of normal data that were misclassified as defective and the amount of defect data that was misclassified as normal, respectively. The parameters FP and FN correspond to type I errors and type II errors, respectively. Let the TP rate (TPR) or recall equal $TP/(TP + FN)$, the TN rate (TNR) equal $TN/(TN + FP)$, and the precision equal $TP/(TP + FP)$. Then, the G-mean and F-measure are defined as follows:

$$G\text{-mean} = \sqrt{TPR \times TNR} \quad (3)$$

$$F\text{-measure} = 2 \cdot \frac{\text{Recall} \times \text{Precision}}{\text{Recall} + \text{Precision}}. \quad (4)$$

The G-mean is the geometric mean of the TPR and the TNR. This measure produces a value within the interval [0, 1], and a higher score indicates better performance. The G-mean produces high scores when errors in both the majority and minority classes are low. The F-measure is the harmonic mean of the precision and recall (i.e., TPR). The precision describes the real defects compared to the number of samples classified as defective, and the recall describes the number of correctly classified defects compared to the number of real defects. Because the precision is defined by $TP/(TP + FP)$, the F-measure score decreases when FP is large.

The receiver operating characteristic (ROC) curve is a visual representation of the tradeoff between the FPR, which is defined as $1 - \text{TNR}$, and the TPR on an (x, y) coordinate [18]. In binary classification problems, the ROC curve can be simplified with the two lines that connect (0, 0) with (FPR, TPR) and (FPR, TPR) with (1, 1) [18]. The AUC describes the area under the ROC curve. The AUC reaches its maximum value of one when $\text{FPR} = 0$ and $\text{TPR} = 1$. However, if $\text{FPR} = 1$ and $\text{TPR} = 0$, then the AUC falls to its minimum value of zero.

C. Results and Analysis

1) *Overall Performance*: We evaluated the performance of the nineteen FD models described in Table I in three data scenarios. Table IV shows the performance results of the FD models. Each entry in the table describes the average performance of a given FD model. As shown in the table, the only model that exhibited good performance on the three performance measures was the IC-FDM algorithm. In particular,

TABLE IV
AVERAGE PERFORMANCE OF FD MODELS IN THE ETCHING PROCESS

Family	FD model	TPR	TNR	Precision	AUC	G-mean	F-measure
F1	SDT	0.46	0.99	0.40	0.73	0.54	0.43
	SSVM	0.61	0.99	0.85	0.80	0.76	0.69
	NDT	0.28	0.99	0.58	0.64	0.34	0.30
	NSVM	0.58	0.99	0.93	0.78	0.74	0.69
	RODT	0.46	0.98	0.36	0.72	0.63	0.40
	ROSVM	0.63	0.98	0.51	0.82	0.78	0.54
F2	BDT	0.14	0.99	0.31	0.57	0.21	0.19
	BSVM	0.53	0.99	0.89	0.76	0.71	0.65
	ABDT	0.29	0.99	0.61	0.64	0.37	0.30
	ABSVM	0.49	0.99	0.71	0.74	0.62	0.55
	RF	0.41	0.99	0.66	0.71	0.50	0.48
	SBDT	0.49	0.94	0.41	0.73	0.66	0.42
F3	SBSVM	0.49	0.98	0.42	0.86	0.57	0.44
	FD-kNN	0.89	0.94	0.26	0.91	0.92	0.33
	PC-kNN	0.46	0.94	0.10	0.70	0.62	0.11
	MD-kNN	0.91	0.97	0.35	0.94	0.94	0.43
F4	IC-FDM	0.89	0.99	0.95	0.94	0.94	0.92
	CS-SVM	0.58	0.99	0.92	0.79	0.74	0.69
	OC-SVM	0.49	0.48	0.03	0.49	0.43	0.06

the model showed a high F-measure score compared to all of the other models. The other models produced low F-measure scores due to the precision index of the F-measure. The precision scores of these models were relatively low compared to that of the IC-FDM algorithm, which implied that the FP (i.e., type I errors) was large in each of the models. These results demonstrated that the F-measure was sensitive to the FP. Note that there are many more normal-class samples than defect-class samples. Thus, the TP could be markedly smaller than that of the FP. To increase the F-measure score, the FP and FN (i.e., type II errors) should be markedly smaller than the TP based on the definitions of the recall (i.e., TPR) and precision. However, the models except for the IC-FDM algorithm failed to decrease both errors, which resulted in low F-measure scores.

The IC-FDM, FD-kNN, and MD-kNN algorithms performed well in terms of the G-mean and AUC. The performance of these models is only slightly affected by the IR because these algorithms can learn classification boundaries without defective data (i.e., one-class learning). These three models produced high TPRs (i.e., low type II error rates) and high TNRs (i.e., low type I error rates). Among the remaining models, the SBSVM, ROSVM, and SSVM algorithms that used an SVM as a classifier recorded an AUC greater than 0.8, showing good performance in the imbalanced scenarios. However, the SBDT, RODT, and SDT algorithms that used a DT as the classifier generally showed lower AUCs. Specifically, the TNRs of the DT-based models were highly similar to those of the SVM-based models; however, the TPRs of the DT-based models were significantly lower compared to those of the SVM-based models. There was a larger FN in the DT compared to the SVM, which signifies that DT was an inferior fault detector compared to SVM in the etching case.

Because the bagging-based FD models (i.e., the BDT and BSVM algorithms) used bootstrap sampling, which selected

TABLE V
PERFORMANCE OF FD MODELS FOR THREE IRS AND RESULTS OF THE WILCOXON TEST (*: P-VALUE>0.05)

Family	FD model	AUC			G-mean			F-measure		
		10 IR	100 IR	1,000 IR	10 IR	100 IR	1,000 IR	10 IR	100 IR	1,000 IR
F1	SDT	0.95*	0.73	0.50	0.95*	0.67	0	0.86	0.43	0
	SSVM	0.94*	0.85	0.63	0.94*	0.83	0.51	0.89	0.78	0.39
	NDT	0.91	0.51	0.50	0.90	0.13	0	0.87	0.03	0
	NSVM	0.92	0.82	0.62	0.91	0.80	0.50	0.91	0.78	0.38
	RODT	0.95*	0.63	0.59	0.95*	0.52	0.42	0.86	0.22	0.11
	ROSVM	0.93	0.83	0.67	0.93	0.82	0.58	0.89	0.37	0.36
F2	BDT	0.70	0.50	0.50	0.64	0	0	0.57	0	0
	BSVM	0.91	0.75	0.64	0.90	0.71	0.52	0.90	0.67	0.39
	ABDT	0.92*	0.51	0.50	0.91*	0.11	0.08	0.88*	0.02	0.01
	ABSVM	0.92	0.78	0.52	0.92	0.75	0.20	0.87	0.72	0.07
	RF	0.95*	0.66	0.50	0.94*	0.57	0	0.94*	0.49	0
	SBDT	0.95*	0.67	0.54	0.95*	0.58	0.44	0.91*	0.33	0
F3	SBSVM	0.92	0.80	0.49	0.92	0.77	0	0.87	0.46	0
	FD-kNN	0.89	0.92	0.95	0.89	0.92	0.95	0.71	0.26	0.04
	PC-kNN	0.54	0.68	0.90	0.35	0.62	0.90	0.16	0.12	0.03
	MD-kNN	0.94*	0.94	0.95	0.94*	0.94	0.95	0.82	0.40	0.08
F4	IC-FDM	0.94	0.94	0.95	0.94	0.94	0.95	0.94	0.93	0.89
	CS-SVM	0.92*	0.82	0.63	0.91*	0.80	0.51	0.91	0.78	0.39
	OC-SVM	0.49	0.46	0.47	0.48	0.43	0.32	0.15	0.02	0

samples from a training dataset with an equal probability, the corresponding models could not assign additional sampling weights to minority-class data during the model training process. As the result, these models exhibited high classification errors (i.e., low TPRs) with respect to the minority class. Furthermore, the performance of the AdaBoost-based FD models (i.e., the ABDT and ABSVM algorithms), which assigned higher sampling weights to misclassified minority class data, was not improved compared to the bagging algorithm-based models because the bootstrap sampling used by the AdaBoost algorithm could not generate new minority class data. Thus, bootstrap sampling presented as an obstacle to resolve the class-imbalance problem. Conversely, the SMOTEBoost algorithm exhibited better performance than the bagging and AdaBoost models. The SMOTEBoost algorithm used the SMOTE sampling technique in each iteration of the AdaBoost algorithm to generate synthetic minority class data; thus, the drawback of the repeated sampling of the minority class data that occurred in the AdaBoost algorithm was avoided. The SMOTEBoost algorithm was the only FD model among the ensemble algorithms that produced a positive result for the class-imbalance problem. The RF algorithm did not exhibit satisfactory performance due to the limitations of using a DT as a classifier.

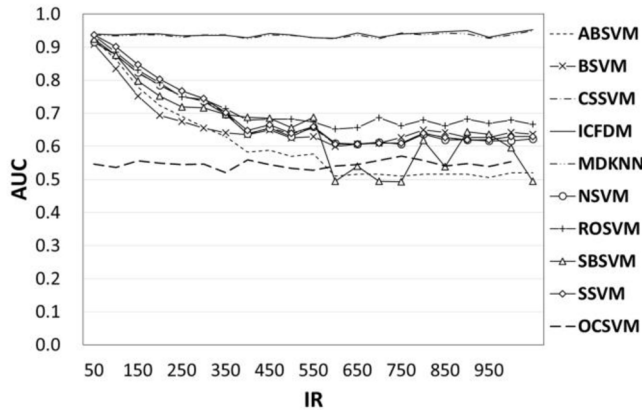
Among the sampling-based methods, the NCL-based FD models (i.e., the NDT and NSVM algorithms) exhibited the worst performances. The performances of the SMOTE-based models (i.e., the SDT and SSVM algorithm) were better than those of the NCL-based models. Interestingly, the performance of random oversampling exhibited the best result among the three sampling algorithms if the F-measure was excluded. This result illustrates that sophisticated sampling techniques do not ensure better conclusions for restrictive circumstances, in

which small amounts of minority class data exist for learning FD models.

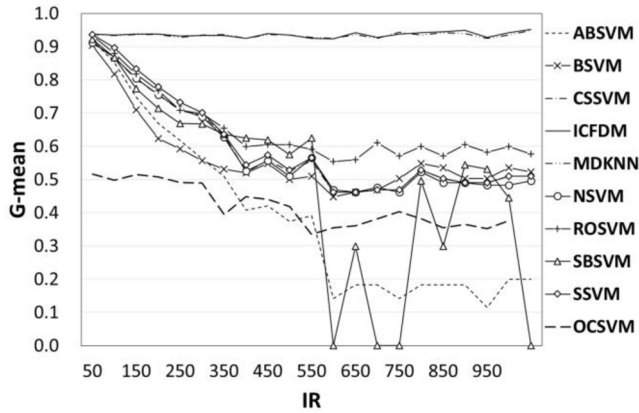
2) *Effect of Class Imbalance*: In this experiment, we assessed the performance of the FD models based on the IR to analyze the influence of the class-imbalance ratio on the performance of the FD models. The results of this experiment are shown in Table V.

Except for the BDT, PC-kNN, and OC-SVM algorithms, the FD models recorded G-mean and AUC scores of approximately 0.9 with an IR of 10. Additionally, except for the BDT, PC-kNN, FD-kNN, and OC-SVM algorithms, the FD models produced F-measure scores greater than 0.8 with the same IR. However, except for the IC-FDM algorithm, the other models exhibited low F-measure scores with an IR of 1,000, which was caused by large FP values. In addition, the difference in performance between the instance-based FD models and the other models grew as the IR increased. In particular, the performances of the MD-kNN and IC-FDM algorithms were markedly superior. The performance of the CS-SVM algorithm was similar to those of the instance-based models for an IR of up to 100. However, its performance declined significantly at an IR of 1,000.

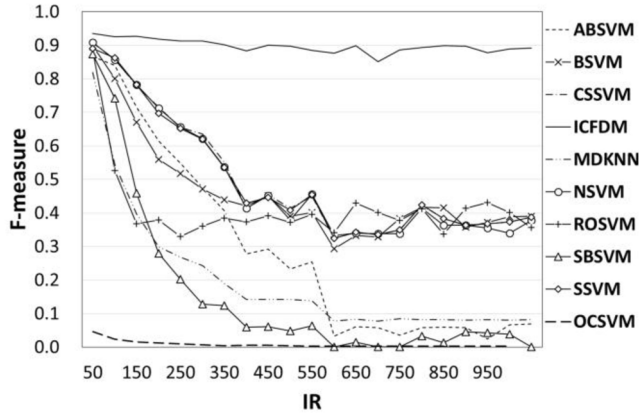
The absolute lack of minority class data is the reason why all of the models except for the instance-based models exhibited poor performances in high-IR situations. When the IR was 1,000 and 1,000 majority class samples were present, there was only one minority class sample. It is difficult to expect learning algorithms to perform satisfactorily when there is an absolute lack of minority class samples, even if algorithms are known to produce high performances in many cases. When the IR was 10, 100 minority class samples were available; thus, the majority of the FD models in Table V produced a score near 0.9 in all three performance measures.



(a) AUC trends.



(b) G-mean trends.



(c) F-measure trends.

Fig. 1. Performance trends based on the class imbalance ratio.

Thus, the performance of the FD models was high when there was a sufficient amount of defective data to explain the causes of most defects. However, if there was a limited amount of defective data available for model training, model training became difficult and performance improvement was limited.

The results of this experiment indicated that the instance-based FD models were effective in high class imbalance situations, and the IC-FDM algorithm showed outstanding performance overall. For preference analysis of the FD models, we conducted the non-parametric Wilcoxon rank-sum test to determine whether IC-FDM shows performance superiority over other algorithms in all three measures. The asterisk

TABLE VI
OPTIMIZED PARAMETER AND PERFORMANCE OF FD MODELS

Family	FD model	AUC	G-mean	F-measure
F1	SDT	0.50 ($cf = 0.1$)	0 ($cf = 0.1$)	0 ($cf = 0.1$)
	NDT	0.50 ($cf = 0.1$)	0 ($cf = 0.1$)	0 ($cf = 0.1$)
	RODT	0.59 ($cf = 0.1$)	0.42 ($cf = 0.1$)	0.11 ($cf = 0.1$)
F2	BDT	0.50 ($cf = 0.1$)	0 ($cf = 0.1$)	0 ($cf = 0.1$)
	ABDT	0.50 ($cf = 0.1$)	0.08 ($cf = 0.1$)	0.01 ($cf = 0.1$)
	RF	0.58 (24 features)	0.15 (24 features)	0.15 (24 features)
	SBDT	0.57 ($cf = 0.7$)	0.44 ($cf = 0.25$)	0.01 ($cf = 0.7$)
F3	FD-kNN	0.95 ($\alpha = 0.95$)	0.95 ($\alpha = 0.95$)	0.66 ($\alpha = 1.0$)
	PC-kNN	0.90 ($\alpha = 0.95$)	0.90 ($\alpha = 0.95$)	0.58 ($\alpha = 1.0$)
	MD-kNN	0.95 ($\alpha = 0.95$)	0.95 ($\alpha = 0.95$)	0.84 ($\alpha = 1.0$)
	IC-FDM	0.95 ($\alpha = 0.99$)	0.95 ($\alpha = 0.99$)	0.89 ($\alpha = 0.99$)
F4	CS-SVM	0.64 ($c = 3$)	0.51 ($c = 1$)	0.39 ($c = 1$)
	OC-SVM	0.54 ($nu = 0.1$)	0.35 ($nu = 0.1$)	0 ($nu = 0.1$)

in Table V indicates that the performance gap between the IC-FDM and test algorithm was not statistically significant at the 95% confidence level. At an IR of 10, the eight algorithms marked with an asterisk showed a performance level that was similar to IC-FDM in terms of the AUC and G-mean measures. However, IC-FDM performed better than all algorithms for IR values greater than 10.

Fig. 1 shows the performance trends of ten FD models based on the IR. The IR started at 10 and increased to 1,000 in increments of 50. As shown in Figs. 1(a) and (b), the trends of the AUC and G-mean were similar, although their scales were different. Based on these trends, we could categorize the FD models into three groups. The top-performance group includes the IC-FDM and MD-kNN algorithms, which produce AUC and G-mean scores of approximately 0.95 at all IRs. The middle-performance group includes the BSVM, CS-SVM, ROSVM, NSVM, and SSVM algorithms. Their AUC and G-mean scores declined as the IR increased. These models produced AUCs of approximately 0.65 and G-means of approximately 0.5 at an IR of 1,000. The bottom-performance group includes the ABSVM, SBSVM, and OC-SVM algorithms, which produced AUCs of approximately 0.5 and G-means under than 0.4 at an IR of 1,000. As shown in Fig. 1, although the OC-SVM scores were not high, the performance trends over varied IR values were stable because the OC-SVM algorithm generated classifiers only with majority class data. In Fig. 1(c), the IC-FDM algorithm produced an F-measure score of approximately 0.9. The other nine models exhibited F-measure trends that were more rapidly decreasing than those in the AUC and G-mean charts.

3) *Parameter Optimization*: In the previous experiment, the parameters of the FD models were determined by referring to the related literature. In this experiment, we optimized the parameters using a greedy search to maximize model performance in the situation with an IR of 1,000. For the DT, the confidence factor (cf) that controls the tree pruning of C4.5 reported by Weka [19] was changed from 0.1 to 0.9 at increments of 0.1. For the CS-SVM algorithm, the SVM tends to reduce the number of misclassifications of the minority class

TABLE VII
PERFORMANCE OF FD MODELS FOR THREE IRs AND RESULTS OF THE WILCOXON TEST (*: P-VALUE>0.05)

Family	FD model	AUC			G-mean			F-measure		
		10 IR	100 IR	1,000 IR	10 IR	100 IR	1,000 IR	10 IR	100 IR	1,000 IR
F1	SDT	0.98*	0.86	0.5	0.98*	0.85	0	0.95*	0.72*	0
	SSVM	0.99	0.84	0.62	0.99	0.82	0.24	0.99	0.79	0.24
	NDT	0.98	0.65	0.5	0.98	0.47	0	0.97	0.40	0
	NSVM	0.99	0.87	0.62	0.99	0.85	0.24	0.99	0.83	0.24
	RODT	0.97	0.80	0.57	0.97	0.76	0.15	0.93	0.59	0.13
	ROSVM	0.99	0.84	0.66	0.99	0.82	0.34	0.99	0.80	0.34
F2	BDT	0.99	0.72	0.5	0.99	0.66	0	0.99	0.59	0
	BSVM	0.99	0.95*	0.5	0.99	0.95*	0	0.99	0.90	0
	ABDT	0.99	0.93*	0.5	0.99	0.92*	0	0.99	0.91	0
	ABSVM	0.99	0.86	0.51	0.99	0.82	0.02	0.99	0.77	0.02
	RF	0.99	0.91	0.54	0.99	0.90	0.08	0.99	0.89	0.08
	SBDT	0.99	0.93*	0.5	0.99	0.92*	0	0.99	0.90	0
F3	SBSVM	0.99	0.97	0.51	0.99	0.97	0.02	0.99	0.95	0.02
	FD-kNN	0.91	0.93	0.96	0.90	0.92	0.93	0.89	0.88	0.86
	PC-kNN	0.97	0.97	0.97	0.97	0.97	0.97	0.79	0.30	0.03
	MD-kNN	0.96	0.98	0.98	0.98	0.98	0.98	0.84	0.38	0.06
F4	IC-FDM	0.99	0.99	0.99	0.99	0.99	0.99	0.96	0.70	0.13
	CS-SVM	0.99	0.87	0.62	0.99	0.85	0.24	0.99	0.83	0.24
	OC-SVM	0.49	0.46	0.47	0.48	0.42	0.31	0.15	0.01	0

samples as the class cost increases. Thus, we defined the cost parameter as 2^c and controlled c to be between -5 and 15 in increments of one. For OC-SVM, the ν parameter was controlled from 0.1 to 0.9 at an increment of 0.1 . For the RF, we changed the number of attributes used to generate each DT. The parameter value in the previous experiment was $\sqrt{p} = \sqrt{24} \approx 5$, where p was the number of original attributes. In this experiment, the RF model was tested by changing the parameter value from a minimum of six to a maximum of 24. The confidence level α was changed to 0.9 , 0.95 and 0.99 for the FD-kNN, PC-kNN, MD-kNN, and IC-FDM algorithms, respectively.

The results of this experiment are shown in Table VI, where the performance improvements due to changes in the parameter are marked in bold letters. Although performance improvements with respect to the AUC and G-mean were observed for the RF, SBDT, and CS-SVM algorithms, the differences were small. Additionally, the performance improvements of the RF, FD-kNN, PC-kNN, and MD-kNN algorithms with respect to the F-measure were insignificant. Furthermore, there was no performance improvement in the sampling-based, bagging-based, or boosting-based FD models that used a DT as a classifier. Thus, the performance of the DT could not be improved by changing the pruning parameter with a large IR.

Supervised machine learning is limited in performance under extremely imbalanced situations, such as an IR of $1,000$, due to the small amount of defective samples that cannot be used to explain the complete patterns of the defect occurrences. However, instance-based FD models, such as the FD-kNN, PC-kNN, and MD-kNN algorithms, determine the classification boundaries with only normal samples and thus may detect the majority of the defects present when the

boundaries are clear. The MD-kNN algorithm produced an F-measure score that approached that of the IC-FDM algorithm. By optimizing parameter α , the FD-kNN and PC-kNN algorithms reached F-measure scores of 0.66 and 0.58 , respectively. However, when compared to the IC-FDM and MD-kNN algorithms, the F-measure scores of the FD-kNN and PC-kNN algorithms were low due to higher FPs.

IV. EXPERIMENT WITH CVD DATA

A. Data and Setup

We conducted an additional performance evaluation test using a field dataset collected from a CVD tool. The dataset consisted of 577 normal wafers and four defective wafers. For each wafer, 40 structural features were extracted from trajectories of the ten process parameters. The features were fed into the FD model training as input variables.

Similar to the etching process, decreases in the trajectories of the ten process parameters were identified as the primary cause of wafer defects in the CVD process. Based on this observation, we introduced simulated normal and defective samples in addition to the real data to create three imbalanced data scenarios with IRs of 10 , 100 , and $1,000$. A total of 50 datasets were prepared for each scenario. The FD models with optimized parameter values specified in Table VI were compared in this experiment.

B. Results and Analysis

The average performance scores of the FD models are presented in Table VII. All FD models showed decreasing trends with respect to the three performance measures as the imbalance ratio increased. In detail, sampling-based and ensemble-based FD models showed slightly better performance than

the other models in terms of the AUC and G-mean at an IR of 10. The FD models using the DT classifier (excluding RODT) produced G-mean scores of zero at an IR of 1,000. These result indicate that DT failed to detect defective wafers; thus, all training samples were classified as normal.

The SMOTEBoost algorithm produced higher performance scores than the other ensemble-based algorithms. These result indicate that the algorithm, which generated artificial minority class samples during the ensemble model training, was effective in resolving class imbalance problems. For the SVM family, CS-SVM only produced good performance scores in all measures at an IR of 10, whereas OC-SVM failed to train adequate classifiers even at the same IR values. These result are derived from the principle of OC-SVM, in that it trains a classifier only with the majority class samples without consideration of possible overlapping with the minority class samples in the data space.

Among the instance-based algorithms, IC-FDM achieved the best AUC and G-mean scores at an IR of 1,000, whereas FD-kNN outperformed IC-FDM with respect to the F-measure. However, except for this case, the Wilcoxon test indicated that IC-FDM demonstrated superior performance when the IR was greater than 100. Overall, the instance-based FD models outperformed the other algorithms at IRs greater than 10. These results are identical to the conclusions of the previous experiment using etching data.

V. CONCLUSION

In the semiconductor manufacturing process, normal wafers are the majority class, and defective wafers are the minority class. Thus, FD models established with class-imbalance learning techniques are likely to successfully detect rare wafer defects. In this study, we constructed nineteen FD models using sampling, ensemble, instance-based, and SVM algorithms and compared the performance of these models using class- imbalanced etching process data and CVD process data. Based on the results of the experiments, we confirmed that instance-based algorithms, such as the IC-FDM, MD-kNN, and FD-kNN algorithms, were fundamentally unaffected by the IR and thus produced good G-mean and AUC scores even in extremely imbalanced situations.

Throughout the analysis of the experiments in this study, the decline in the performance of the FD models was found to be caused by the lack of defective samples and by the lack of diversity in the defective samples. If wafer defects are caused by various process parameters, and defective samples available for model training are located in a small subregion of the space constructed by process parameters, the FD models only learn defective patterns in the subregion; this produces a low performance with respect to model verification. Thus, defective data for model training should include most defective patterns, even if they only occur at a low frequency. For the FD cases in which the number of defective samples is extremely small or nonexistent, instance-based models would be effective for FD.

REFERENCES

- [1] T. Lee and C. O. Kim, "Statistical comparison of fault detection models for semiconductor manufacturing processes," *IEEE Trans. Semicond. Manuf.*, vol. 28, no. 1, pp. 80–91, Feb. 2015.
- [2] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, no. 1, pp. 321–357, 2002.
- [3] J. Laurikkala, "Improving identification of difficult small classes by balancing class distribution," in *Proc. Conf. Artif. Intell. Med. Europe, Cascais, Portugal*, 2001, pp. 63–66.
- [4] L. Breiman, "Bagging predictors," *Mach. Learn.*, vol. 24, no. 2, pp. 123–140, 1996.
- [5] R. E. Schapire, "The strength of weak learnability," *Mach. Learn.*, vol. 5, no. 2, pp. 197–227, 1990.
- [6] L. Breiman, "Random forest," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.
- [7] N. V. Chawla, A. Lazarevic, L. O. Hall, and K. W. Bowyer, "SMOTEBoost: Improving prediction of the minority class in boosting," in *Proc. 7th Eur. Conf. Principles Pract. Knowl. Discov. Databases*, 2003, pp. 107–119.
- [8] Q. P. He and J. Wang, "Fault detection using the k-nearest neighbor rule for semiconductor manufacturing processes," *IEEE Trans. Semicond. Manuf.*, vol. 20, no. 4, pp. 345–354, Nov. 2007.
- [9] Q. P. He and J. Wang, "Large-scale semiconductor process fault detection using a fast pattern recognition-based method," *IEEE Trans. Semicond. Manuf.*, vol. 23, no. 2, pp. 194–200, May 2010.
- [10] G. Verdier and A. Ferreira, "Adaptive Mahalanobis distance and k-nearest neighbour rule for fault detection in semiconductor manufacturing," *IEEE Trans. Semicond. Manuf.*, vol. 24, no. 1, pp. 59–68, Mar. 2011.
- [11] J. Kwak, T. Lee, and C. O. Kim, "An incremental clustering-based fault detection algorithm for class-imbalanced process data," *IEEE Trans. Semicond. Manuf.*, vol. 28, no. 3, pp. 318–328, Aug. 2015.
- [12] P. Cao, D. Zhao, and O. Zaiane, "An optimized cost-sensitive SVM for imbalanced data learning," in *Advances in Knowledge Discovery and Data Mining*, Berlin, Germany: Springer-Verlag, 2013, pp. 280–292.
- [13] B. Schölkopf, J. C. Platt, J. C. Shawe-Taylor, A. J. Smola, and R. C. Williamson, "Estimating the support of a high-dimensional distribution," *Neural Comput.*, vol. 13, no. 7, pp. 1443–1471, 2001.
- [14] M. Galar, A. Fernandez, E. Barrenechea, H. Bustince, and F. Herrera, "A review on ensembles for the class imbalance problem: Bagging-, boosting-, and hybrid-based approaches," *IEEE Trans. Syst., Man, Cybern. C, Appl. Rev.*, vol. 42, no. 4, pp. 463–484, Jul. 2012.
- [15] Y. Freund and R. E. Schapire, "A decision-theoretic generalization of on-line learning and an application to boosting," *J. Comput. Syst. Sci.*, vol. 55, no. 1, pp. 119–139, 1997.
- [16] Eigenvector Res. Inc. (1999). *Metal Etch Data for Fault Detection Evaluation*. [Online]. Available: <http://software.eigenvector.com/Data/Etch/index.html>
- [17] RDevelopment Core Team, *R: A Language and Environment for Statistical Computing*, Vienna, Austria, 2009. [Online]. Available <http://www.R-project.org>
- [18] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognit. Lett.*, vol. 27, no. 8, pp. 861–874, 2006.
- [19] M. Hall *et al.*, "The WEKA data mining software: An update," *ACM SIGKDD Explor. Newslett.*, vol. 11, no. 1, pp. 10–18, 2009.



Taehyung Lee received the B.S. degree in industrial engineering from Hansung University, South Korea, in 2004, and the M.S. and Ph.D. degrees in industrial engineering from Yonsei University, South Korea, in 2008 and 2015, respectively. He is currently a Senior Engineer with Hyundai Motor Group. His current research interest is big data analysis for manufacturing.



Ki Bum Lee received the B.S. degree in industrial engineering from Yonsei University, South Korea, in 2014, where he is currently pursuing the Ph.D. degree in industrial engineering. His current research interest is machine learning for manufacturing.



Chang Ouk Kim received the B.S. and M.S. degrees in industrial engineering from Korea University, Seoul, South Korea, in 1988 and 1990, respectively, and the Ph.D. degree in industrial engineering from Purdue University, West Lafayette, IN, USA, in 1996. He is a Professor with the Department of Information and Industrial Engineering, Yonsei University, South Korea. His current research interest is data science for manufacturing.